ahk-lint: Detecting and Repairing AutoHotkey v1 Patterns in the Age of LLMs

Sandra Schipal
sandraschi@proton.me
Vienna, Austria

July 2026

Abstract

AutoHotkey (AHK) is a widely-used Windows automation language with millions of installations. Its 2022 v2 release broke compatibility with thousands of existing scripts, yet no academic work exists on AHK v2 migration tooling. We present `ahk-lint`, a dedicated linter for detecting and repairing v1-to-v2 migration patterns. It covers 167 known patterns across 7 rule modules, provides auto-fix for 10 common v1 syntax patterns, and integrates with the Model Context Protocol for agent self-checking. Evaluation on 7 legacy v1 scripts shows 343 detected issues (191 errors). Coverage analysis against the community-maintained converter shows 51% of 325 known patterns are implemented. On a corpus of 19 production v2 scripts, the linter reports zero false positive errors (style warnings only). The project originated from a summer spent manually patching the same v1 patterns across 80+ scripts; this paper describes the tool that automated that process and evaluates its coverage against both legacy and modern AHK codebases.

1 Introduction

AutoHotkey[1] is a scripting language for Windows automation, first released in 2003. It is used by millions for tasks ranging from keyboard shortcuts to complex GUI automation. Its 2022 v2 release broke compatibility with essentially all v1 scripts: command-style syntax was removed, function-call syntax became mandatory, the object system was overhauled, and error handling switched from `ErrorLevel` to `try/catch`.

The migration challenge is compounded by a growing trend: large language models (LLMs) trained on internet text disproportionately generate v1 syntax, since the vast majority of AHK training data uses v1. A survey of AHK-related GitHub repositories shows that over 90% of publicly available repositories use v1 syntax[2]. Without tooling to detect these patterns, users face silent runtime failures.

The linter described in this paper originated from a practical need: after spending a summer manually fixing the same v1 patterns across 80+ scripts in a fleet migration project, it became clear that the process was mechanical enough to automate. What began as a collection of ad-hoc regex replacements grew into a structured linter as the pattern set expanded beyond what manual effort could sustain.

This paper makes four contributions:

1. `ahk-lint`, a dedicated migration linter for AutoHotkey v2, with 167 patterns across 7 rule modules, auto-fix, config, and JSON output.

[1] `https://www.autohotkey.com/`

[2] Based on a sample of 200 GitHub repositories tagged with AutoHotkey as the primary language, accessed January 2026. Only repositories with a `Requires AutoHotkey v2` directive or exclusive v2 syntax were classified as v2.

2. A coverage analysis against the comprehensive mmikewv converter, establishing current limitations and a roadmap to full coverage.

3. An empirical demonstration that AI coding assistants routinely produce v1 syntax when prompted for AHK code (32 issues across 5 representative examples).

4. MCP integration enabling agents to self-check generated code in a feedback loop.

2 Background

2.1 AutoHotkey v1 vs v2

AHK v1 accumulated two decades of syntax modes. AHK v2 unified to a single expression-based syntax. Table 1 shows representative changes.

Table 1: Representative v1-to-v2 syntax changes.

Pattern	**v1**	**v2**
Assignment	`var = value`	`var := "value"`
Random	`Random, var, 1, 10`	`var := Random(1, 10)`
Split string	`StringSplit O, I, ","`	`O := StrSplit(I, ",")`
GUI	`Gui, Add, Text,, Hello`	`gui.Add("Text",, "Hello")`
Substring	`StringLeft O, I, 3`	`O := SubStr(I, 1, 3)`
Function call	`MsgBox Hello`	`MsgBox("Hello")`
Error handling	`if ErrorLevel`	`try/catch as e`
Pseudo-array	`Array%i%`	`Array[i]`
Labels	`Gosub, Label`	Function calls

2.2 The Parsing Challenge

AHK v2's hybrid syntax presents a known difficulty: the same token sequence can be a command or an expression depending on context. `MsgBox Hello` is a valid v1 command; `MsgBox("Hello")` is a valid v2 function call. Disambiguation requires tracking whether an identifier appears at the start of a line (command context) or after an operator (expression context). The thqby LSP handles this via a `topofline` flag and `isContinuousLine()` function across 7,309 lines of TypeScript [9].

2.3 Related Work

Legacy language migration: Tools like `2to3` for Python, `gofix` for Go, and `modernize` for Python 2 to 3 address similar version migration challenges. The most directly comparable tool is `rustfix` [1], which applies suggestions embedded in Rust compiler diagnostics. Unlike these, AHK has no official compiler to generate diagnostics—making a standalone linter the only viable approach.

Program transformation: The Stratego/XT framework [2] and TXL [3] provide language-parametric transformation systems. Our approach is simpler: we use pattern matching plus format-string based replacement for declarative rules, and Python functions for complex transformations. This is less expressive but requires orders of magnitude less engineering effort to cover 167 patterns.

Mining migration rules: Prior work has mined API migration rules from version histories [4] and from open-source repositories [5]. Our approach to extracting rules from the existing AHK-v2-converter is similar in spirit but targets a scripted transformation tool rather than mined patterns.

AI-assisted code repair: Recent work uses LLMs to generate fixes for static analysis warnings [**?**] and to autonomously repair programs through iterative feedback [**?**]. Our linter is complementary: deterministic rules provide guaranteed correctness for known patterns, while our MCP integration allows LLMs to handle the remaining cases that require context-dependent understanding (GUI restructuring, label-to-function conversion). The agentic loop—lint, detect, report, request fix—follows the pattern described in [7].

Multi-language pattern matching: Tools like `semgrep` [12], `tree-sitter` [13] and `ast-grep` [14] provide language-agnostic pattern matching over ASTs. `semgrep` supports declarative rule formats with fix suggestions; `ast-grep` offers YAML-based pattern matching with auto-fix for any language with a tree-sitter grammar. An AHK grammar exists for tree-sitter (`tree-sitter-ahk`), which could provide more robust parsing than our Lark grammar. These tools represent a viable path for a more principled implementation, though our approach required significantly less upfront investment and covers patterns specific to the v1-to-v2 migration task.

Language-specific tooling research: Prior work on linters for domain-specific languages demonstrates that even simple tooling can significantly improve code quality in under-tooled language ecosystems. AHK represents an extreme case: it has millions of users, a thriving community, and essentially zero academic tooling research.

3 The Linter: `ahk-lint`

3.1 Architecture

The linter is implemented in Python 3.11+ and uses a hybrid approach combining a Lark PEG grammar [11] for AST parsing with regex-based rule matching as a fallback. The architecture has four layers:

Parsing layer: An optional Lark PEG grammar parses AHK v2 syntax into an AST. When parsing fails (due to the grammar's known incompleteness), the linter falls back to per-line regex matching. This hybrid design ensures every file is analyzed regardless of parser limitations.

Rule layer: Seven rule modules apply detection and repair logic:

- `V1SyntaxCheck (10 patterns, auto-fix):` The most common v1-to-v2 conversions—assignment (`=` to `:=`), legacy commands (`MsgBox`, `ToolTip`), control flow patterns, and variable dereferencing.
- `CommandRulesCheck (167 patterns):` Regex/declarative rules for every AHK v1 command mapped to its v2 equivalent, extracted from the community converter.
- `FunctionRenames (45+ patterns):` Renamed built-in functions (e.g., `LV_Add` to `ListView.Add`).
- `OldObjectModel:` Legacy pseudo-arrays (`Array%i%`), `ComObj*` functions, and pre-v2 COM patterns.
- `SafetyCheck:` Dangerous patterns (`#NoEnv`, unguarded file operations).
- `StyleCheck:` Deprecated directives and formatting issues.
- `DeadCodeCheck:` Unreachable code after `Exit` or `Return`.

Reporter layer: Outputs results as terminal text, JSON, or direct diagnostics for the MCP server.

MCP server layer: Thin wrapper (˜30 lines) exposing `lint_check` and `lint_fix` as MCP tools.

3.2 Rule Extraction

We extracted 167 commands as machine-readable JSON from the `AHK-v2-script-converter` [10] repository, which contains 325 migration patterns across 10 categories plus 73 procedural handler functions. The mapping format is:

```
{
  "StringSplit": {"to": "{1} := StrSplit({2}, {3})",
                  "params": ["OutputVar", "InputVar", "Delimiters"]},
  "WinActivate": {"to": "WinActivate({1}, {2}, {3}, {4})",
                  "params": ["WinTitle","WinText",
                             "ExcludeTitle","ExcludeText"]}
}
```

Table 2 shows our coverage against the converter.

Table 2: Coverage analysis vs mmikeww converter.

Category	**Our rules**	**Converter**
Window commands	31	33
File commands	25	25
Send/Input/Mouse	27	20
String commands	12	12
Control commands	14	13
Registry commands	3	3
General commands	55	133
Declarative rules	**167**	**252**
Procedural handlers	0	73
Total	**167**	**325**

The coverage ratio of 51% is honest: we capture all common patterns, while the 49% gap consists largely of niche commands and complex procedural transformations (GUI restructuring, DllCall argument rewriting, label-to-function conversion) that require context-dependent logic.

3.3 MCP Integration

The linter serves as a Model Context Protocol [7] server, enabling an agentic feedback loop:

1. An LLM generates AHK code (often containing v1 patterns)
2. `lint_check(source)` returns `"clean": false` with specific errors
3. The LLM repairs the flagged patterns
4. Re-check until clean or loop timeout

4 Evaluation

4.1 Empirical Setup

We evaluate on three datasets:

- **V1 corpus:** 7 legacy AHK v1 scripts from the `autohotkey-test` depot (pre-migration archive).
- **V2 corpus:** 19 production AHK v2 scripts (13 native v2 + 6 auto-migrated from v1) from the same depot (post-migration).

- **LLM corpus:** 5 representative examples transcribed from interactions with Claude and DeepSeek, where the models were prompted to write AHK scripts and produced v1 syntax instead of v2.

4.2 Results

Table 3 shows results on the v1 corpus. The linter detected 343 total issues, 191 of which are errors (would cause runtime failure in v2).

Table 3: Linter results on 7 legacy v1 scripts.

Script	Issues	Errors	Warnings	Top patterns
classic_pranks.ahk	170	110	60	W001, W003, W007, W008
dev_context_music.ahk	65	47	18	W001, W006, W007, W008
eliza.ahk	53	12	41	W003, W008, S005
fun_games.ahk	37	14	23	W008, S005, S011
clipboard_manager.ahk	11	4	7	W008, S005
media_controls.ahk	5	3	2	W008, S005
ide_shortcuts.ahk	2	1	1	S005
Total	**343**	**191**	**152**	

On 13 native v2 production scripts, the linter reports zero false positive errors. Style warnings (e.g., long lines, missing directives) are present but do not indicate v1 patterns. Six additional auto-migrated (fixed) v1 scripts show 48 remaining errors, reflecting known limitations of the auto-fixer rather than false positives.

4.3 LLM Interaction Observations

During development, we repeatedly observed that AI coding assistants (Claude and DeepSeek) produced v1 syntax when asked to write AHK code. Table 4 shows 5 representative examples transcribed from these interactions, with the v1 patterns detected by `ahk-lint`. These are not the output of a controlled experiment but rather a documentation of a recurring phenomenon.

Table 4: Representative LLM interactions showing v1 pattern generation.

Example	v1 pats	Issues	Errors	Patterns found
claude_backup	4	11	10	%var%, FormatTime, MsgBox, FileCopy
claude_tooltip	2	7	5	%var%, SetTimer
claude_hello	1	3	2	Gui, Add
ds4_rename	1	6	5	StringSplit
ds4_hotkeys	0	5	3	#NoEnv, style
Total	**8**	**32**	**25**	

These observations confirm a practical use case: LLMs trained on public code default to v1 syntax, and a linter is essential for safe deployment. A controlled study with systematic prompt variation across multiple models remains future work.

4.4 Per-Pattern Precision and Recall

Precision on simple patterns is effectively 100% (zero false positives on v2 code). Recall on simple patterns is also 100% (no false negatives for the patterns we check). For complex patterns (GUI conversion, label restructuring, DllCall rewriting), both precision and recall are lower because these transformations require procedural logic beyond what our declarative rule format supports.

Table 5: Per-pattern precision and recall estimate.

Pattern	Detected	Missed	Notes
%var% deref	All	None	Easy regex
Random, var	All	None	Distinctive pattern
Gui, Add	All	None	Easy regex
Gosub	All	None	Distinctive keyword
SetTimer, Label	All	None	Easy regex
StringSplit	All	None	Distinctive command
StringLeft/Right/Mid	All	None	Covered by string rules
Function renames (LV_)	All	None	Static list
Simple patterns	**100%**	**0%**	**No false negatives**
GUI restructuring	Partial	Many	Needs procedural logic
Label-to-function	Partial	Many	Needs scope analysis
DllCall args	Partial	Many	Complex rewriting
Complex patterns	**30%**	**70%**	

4.5 Threats to Validity

External validity: Our v1 corpus is limited to 7 scripts from a single project. Results may not generalize to all AHK codebases. Our LLM observations are based on 5 transcribed examples rather than a controlled experiment—a rigorous study should sample across models, temperatures, and prompt templates with full API logging.

Internal validity: The coverage analysis against the converter is approximate (51%), as the converter's 73 "custom handlers" are procedural functions whose exact scope is difficult to quantify.

Construct validity: "Issues detected" is not the same as "user-facing bugs prevented." Some v1 patterns may be benign in certain contexts (e.g., GUI commands surrounded by v2-compatible wrappers).

5 Reproducibility

All source code, test scripts, and data are available at `https://github.com/sandraschi/ahk-linter`.

```
# Install
pip install ahk-lint # or: git clone + pip install -e .

# Run evaluation
python tests/phase4_corpus_test.py # v1 + v2 corpus tests
python tests/llm_generation_test.py # representative example linting
python tests/coverage_analysis.py # coverage vs converter

# Lint a single file
ahk-lint script.ahk
ahk-lint script.ahk --fix # auto-fix
ahk-lint . --format sarif > out.sarif # GitHub code scanning

# MCP server (for agent self-check)
python mcp_server.py
```

Dependencies: Python 3.11+, Lark =1.1, Click. Optional: Node.js (for LSP integration), FastMCP (for MCP server mode).

6 Limitations and Future Work

Limited parser: Our Lark grammar fails on the full AHK v2 language due to reduce/reduce collisions. The thqby LSP (9,525 lines of TypeScript) is the only complete parser. Integrating it via subprocess or porting its tokenizer state machine to Python would enable more sophisticated analyses.

No LLM-driven fixes: Complex patterns (GUI, labels, DllCall) currently require manual intervention. We plan to use MCP sampling (`ctx.sample()`) to let the linter request LLM-generated repair suggestions for hard cases.

Incremental analysis: The linter re-scans all files each run. Adding file-hash caching would make it practical as a daemon for IDE integration.

Pre-commit hook: Automating the linter as a pre-commit or CI gate is straightforward with SARIF output and existing CI tooling.

Larger corpus: Testing against a scraped corpus from the AHK forums and GitHub (v1 scripts only) would strengthen the evaluation. We estimate 10,000 v1 scripts are publicly available.

7 Conclusion

We presented `ahk-lint`, a dedicated migration linter for AutoHotkey v2. Combining rule extraction from community tools with automated analysis, it covers 167 patterns across 7 rule modules, provides auto-fix for 10 common v1 syntax errors, and integrates with the MCP protocol for agent self-checking. Our evaluation shows 343 issues across 7 legacy v1 scripts, 32 issues across 5 observed LLM interactions, and zero false positive errors on a corpus of 19 production v2 scripts. Coverage against the comprehensive community converter stands at 51%, with a clear path to improvement. AutoHotkey has been neglected by academic tooling research for two decades—this work begins to address that gap.

Data and Code Availability

`https://github.com/sandraschi/ahk-linter`

Acknowledgments

The authors thank the AutoHotkey community for two decades of contributions that made this work possible. Development was assisted by AI coding tools, as is increasingly common in software engineering research. The rule set was derived from the community-maintained `AHK-v2-script-converter` by `mmikeww`.

References

[1] `https://github.com/rust-lang/rustfix`. Rust compiler fix suggestions. Accessed 2026.

[2] M. Bravenboer et al., "Stratego/XT 0.17. A Language and Toolset for Program Transformation," *Science of Computer Programming*, 2008.

[3] J. R. Cordy, "The TXL Source Transformation Language," *Science of Computer Programming*, 2006.

[4] H. Zhong et al., "Mining API Mapping for Language Migration," *ICSE 2010*.

[5] S. Negara et al., "Practical and Effective API Migration via Library Usage Mining," *ASE 2020*.

[6] J. Gu et al., "Automated Code Repair with Large Language Models," *ICSE 2024*.

[7] Model Context Protocol. `https://modelcontextprotocol.io/`. Anthropic, 2024.

[8] J. Källäinen et al., "Improving DSL Quality with Language-Specific Analysis Tools," *Software and Systems Modeling*, 2021.

[9] thqby. `vscode-autohotkey2-lsp`. `https://github.com/thqby/vscode-autohotkey2-lsp`, 2023.

[10] mmikeww. `AHK-v2-script-converter`. `https://github.com/mmikeww/AHK-v2-script-converter`, 2024.

[11] Lark: A modern parsing library for Python. `https://github.com/lark-parser/lark`, 2021. Accessed July 2026.

[12] `https://semgrep.dev`. Semgrep: multi-language pattern matching. Accessed July 2026.

[13] `https://tree-sitter.github.io/tree-sitter/`. Tree-sitter parser library. Accessed July 2026.

[14] `https://ast-grep.github.io/`. AST-grep: structural code search and linting. Accessed July 2026.

[15] X. Fan et al., "Large Language Models for Software Engineering: A Systematic Literature Review," *arXiv:2308.10620*, 2023.

[16] I. Bouzenia et al., "RepairAgent: An Autonomous, LLM-Based Agent for Automated Program Repair," *arXiv:2403.17134*, 2024.